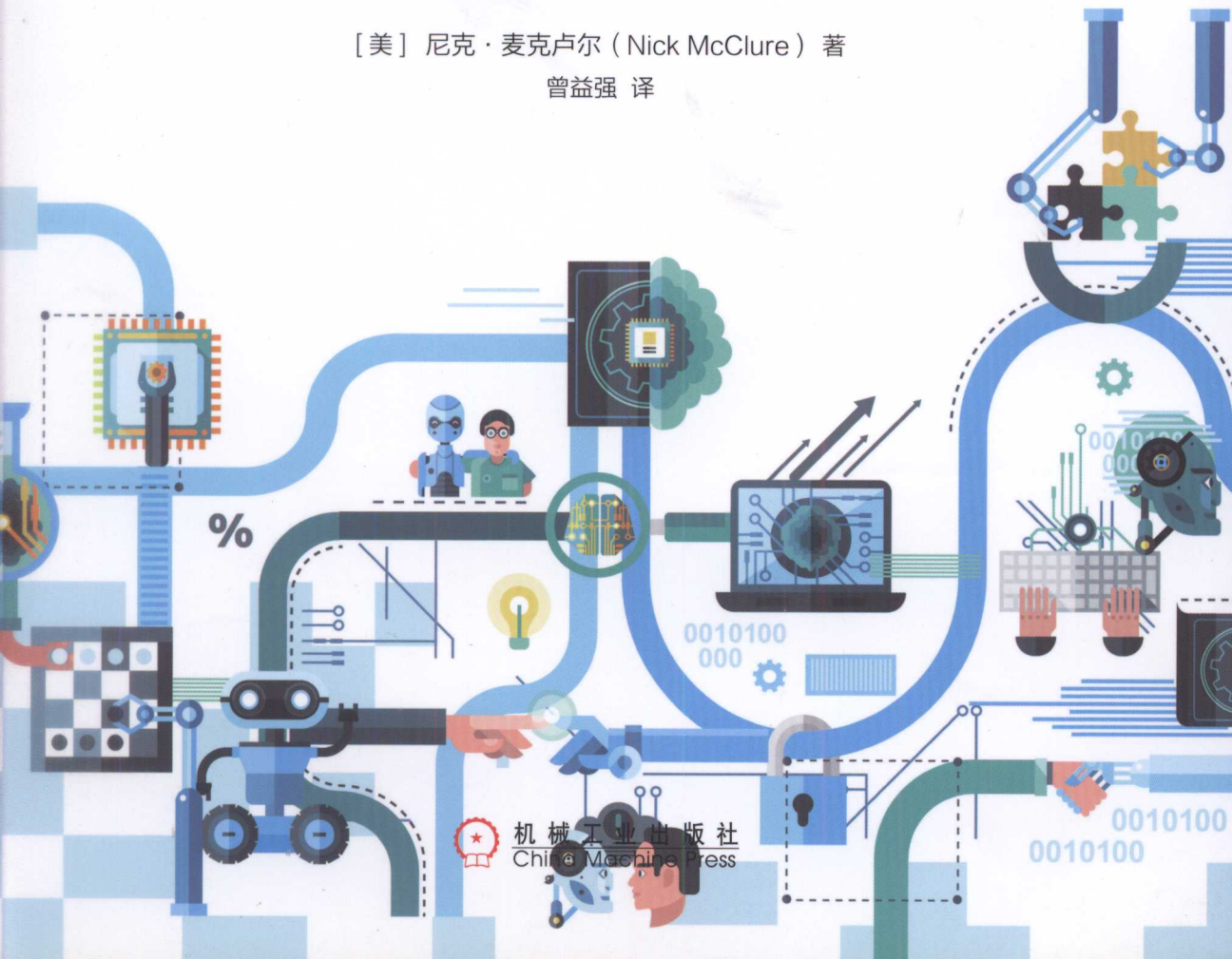


TensorFlow Machine Learning Cookbook

TensorFlow机器学习 实战指南

[美] 尼克·麦克卢尔 (Nick McClure) 著

曾益强 译

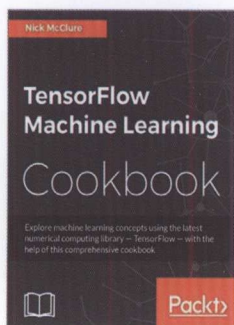


机械工业出版社
China Machine Press

内容简介

本书由资深数据科学家撰写，从实战角度系统讲解TensorFlow基本概念及各种应用实践。真实的应用场景和数据，丰富的代码实例，详尽的操作步骤，带你由浅入深系统掌握TensorFlow机器学习算法及其实现。

全书共11章，第1章介绍TensorFlow的基本概念；第2章介绍如何在计算图中连接算法组件，创建一个简单的分类器；第3章重点介绍如何使用TensorFlow实现各种线性回归算法；第4章介绍支持向量机（SVM）算法；第5章介绍如何使用数值度量、文本度量和归一化距离函数实现最近邻域算法；第6章讲述如何使用TensorFlow实现神经网络算法；第7章阐述TensorFlow实现的各种文本处理算法；第8章卷积神经网络算法；第9章解释在TensorFlow中如何实现递归神经网络（RNN）算法；第10章介绍TensorFlow产品级用例；第11章展示TensorFlow如何实现遗传算法、k-means算法和求解常微分方程（ODE）。



原书封面

智能系统与技术丛书

图书在版编目(CIP)数据

TensorFlow机器学习指南(第2版) / (美)尼克·麦克卢尔(Nick McClure)著;曾益强译.

—北京:机械工业出版社,2017.9

(智能系统与技术丛书)

书名原文:TensorFlow Machine Learning Cookbook

ISBN 978-7-111-57048-9

I.T—II.①美…②曾… III.①人工智能—算法—指南 IV.TP18

中国版本图书馆CIP数据核字(2017)第218381号

TensorFlow Machine Learning Cookbook

TensorFlow机器学习 实战指南

[美] 尼克·麦克卢尔 (Nick McClure) 著

曾益强 译



机械工业出版社
China Machine Press

图书在版编目 (CIP) 数据

TensorFlow 机器学习实战指南 / (美) 尼克·麦克卢尔 (Nick McClure) 著; 曾益强译.

—北京: 机械工业出版社, 2017.9

(智能系统与技术丛书)

书名原文: TensorFlow Machine Learning Cookbook

ISBN 978-7-111-57948-9

I. T… II. ①尼… ②曾… III. 人工智能—算法—研究 IV. TP18

中国版本图书馆 CIP 数据核字 (2017) 第 218381 号

本书版权登记号: 图字 01-2017-2021

Nick McClure: TensorFlow Machine Learning Cookbook (ISBN: 978-1-78646-216-9).

Copyright © 2017 Packt Publishing. First published in the English language under the title
“TensorFlow Machine Learning Cookbook”.

All rights reserved.

Chinese simplified language edition published by China Machine Press.

Copyright © 2017 by China Machine Press.

本书中文简体字版由 Packt Publishing 授权机械工业出版社独家出版。未经出版者书面许可, 不得以任何方式复制或抄袭本书内容。

TensorFlow 机器学习实战指南

出版发行: 机械工业出版社 (北京市西城区百万庄大街 22 号 邮政编码: 100037)

责任编辑: 陈佳媛

责任校对: 李秋荣

印 刷: 北京市荣盛彩色印刷有限公司

版 次: 2017 年 9 月第 1 版第 1 次印刷

开 本: 186mm×240mm 1/16

印 张: 18

书 号: ISBN 978-7-111-57948-9

定 价: 69.00 元

凡购本书, 如有缺页、倒页、脱页, 由本社发行部调换

客服热线: (010) 88379426 88361066

投稿热线: (010) 88379604

购书热线: (010) 68326294 88379649 68995259

读者信箱: hzit@hzbook.com

版权所有·侵权必究

封底无防伪标均为盗版

本书法律顾问: 北京大成律师事务所 韩光 / 邹晓东

THE TRANSLATOR'S WORDS

译者序

2017年3月底，华章公司的编辑邀请我翻译这本书。当时收到原书目录和样章时，大体浏览了一遍，感觉翻译难度不大。因为 TensorFlow 比较火，加上自身对机器学习及其算法有一定功底，前期也翻译了不少国外优秀的技术文章（可参见公众号：神机喵算），加之国内可学习的 TensorFlow 资料太少，所以我希望做出一些努力来帮助对 TensorFlow 感兴趣的读者。

Google 公司开发的 TensorFlow 深度学习库因其简单易学、应用场景广泛已经快成为各家公司开展人工智能研究的标配了。TensorFlow 采用数据流图进行数值计算。节点代表计算图中的数学操作，计算中的边表示多维数组，即张量。TensorFlow 灵活的架构使其可以在多种设备（台式机、服务器或移动设备）的 CPU 或者 GPU 上进行计算。自从 TensorFlow 诞生以来，其开发版更新和功能优化非常快，当前已经发布到 1.2.0。并且基于 TensorFlow 开发的深度学习库也越来越多，其中比较优秀的是 Keras。Keras 是基于 TensorFlow 或者 Theano 的，由 Python 编写的高级神经网络 API，并且 TensorFlow 也提供支持 Keras 的 API。

本书详细讲解了 TensorFlow 的方方面面，毫不夸张地说，如果读者能够坚持踏踏实实做完本书所有实战项目，则基本可以开始使用 TensorFlow 实际工作。最后本书还给出了 TensorFlow 产品级应用的最佳实践，以及扩展用法。

总之，本书适合广大对 TensorFlow 感兴趣的初中级读者。随着 AI 的兴起，会有越来越多的读者学习 TensorFlow，希望本书能帮到大家。如果想进一步学习，那就要多看机器学习算法相关的书籍或者论文，并把 TensorFlow 的源代码研读几遍。

最后，感谢家人和朋友的帮助和支持。由于本人水平有限，加之翻译时间仓促，书中难免会出现错误。读者可通过本人公众号——神机喵算，反馈问题，发现问题后，我一定会虚心接受批评并立即改正，并实时在公众号更新勘误，避免其他读者再入“坑”。

曾益强

2017年6月

图书在版编目(CIP)数据

TensorFlow 机器学习实战指南 / (美) 尼克·麦克卢尔 (Nick McClure) 著; 曾桂强译.

—北京: 机械工业出版社, 2017.9

(智能系统与技术丛书)

书名原文: TensorFlow Machine Learning Cookbook

4. Q ABOUT THE AUTHOR INT

ISBN 978-7-111-57945-9

作者简介

LT... IL D... 2... RI... IV, TPIR

中国版本图书馆CIP数据核字(2017)第218381号

本书版权登记号: 图字 01-2017-2021

Nick McClure, 资深数据科学家, 目前就职于美国西雅图 PayScale 公司, 曾经在 Zillow 公司和 Caesar's Entertainment 公司工作, 获得蒙大拿大学和圣本尼迪克与圣约翰大学的应用数学专业学位。

他热衷于数据分析、机器学习和人工智能。Nick 有时会把想法写成博客 (<http://fromdata.org/>) 或者发推特 (@nfmccclure)。

感谢父母, 他们总是鼓励我追求知识。也感谢朋友和同事能够给出很好的建议。本书的完成得益于开源社区的不懈努力, 以及 TensorFlow 相关项目的良好文档说明。

这里, 要特别感谢 Google 公司 TensorFlow 开发人员。他们给出了优秀的官方文档、教程和示例。

TensorFlow 机器学习

TensorFlow 机器学习

TensorFlow 机器学习

TensorFlow 机器学习

TensorFlow 机器学习

TensorFlow 机器学习

TensorFlow 机器学习

TensorFlow 机器学习

TensorFlow 机器学习

TensorFlow 机器学习

TensorFlow 机器学习

Word2Vec 和 Doc2Vec 用这些方法来做预测。

第 8 章扩展神经网络算法，说明如何借助卷积神经网络（CNN）算法在图像上应用神经网络算法。我们展示如何构建一个简单的 CNN 进行 MNIST 数字识别，并扩展到 CIFAR-10 任务中的彩色图片，也通过 AlexNet 模型实现 ImageNet 的图像识别模型。本章末尾

ABOUT THE REVIEWERS

详细解释 TensorFlow 实现的模仿人脑的神经网络算法。

审校者简介

第 9 章解释在 TensorFlow 中如何实现递归神经网络（RNN）算法，展示如何进行垃圾短信预测和在莎士比亚文本样本集上扩展 RNN 模型生成文本。接着训练 Seq2Seq 模型实现德语-英语的翻译，本章末尾展示如何用变长 RNN 模型进行地址记录匹配。

Chetan Khatri，具有 5 年工作经验的数据科学研究者。他现在是印度 Accion Labs 公司技术部门的负责人，曾就职于印度手游巨头 Nazara Games 公司，领导负责游戏与电信业务。

他在 KSKV Kachchh 大学计算机科学和数据分析专业取得硕士学位，致力于数据科学、机器学习、AI 和 IoT 等方面的学术和会议演讲交流。他在学术研究和工业实践两方面都有特长，所以在排除两者间的隔阂方面有不错的成就。他是 Kachchh 大学多门课程的合作者，比如数据分析、IoT、机器学习、AI 和分布式数据库。他也是 Python 社区（PyKuth）的建立者之一。（<https://www.python.org/downloads/>）编写的。大部分章节需要访问及网络中下载

目前，他正致力于智能 IoT 设备与机器学习、增强学习和分布式计算方面的结合。

感谢 Kachchh 大学计算机科学学院 Devji Chhanga 教授引导我走上数据分析研究的道路。

感谢 Shweta Gorania 教授介绍遗传算法和神经网络算法。

最后，感谢家人的支持。

前言

2015 年 11 月，Google 公司开源 TensorFlow，随后不久 TensorFlow 成为 GitHub 上最受欢迎的机器学习库。TensorFlow 创建计算图、自动求导和定制化的方式使得其能够很好地解决许多不同的机器学习问题。

本书介绍了许多机器学习算法，将其应用到真实场景和数据中，并解释产生的结果。

本书的主要内容

第 1 章介绍 TensorFlow 的基本概念，包括张量、变量和占位符；同时展示了在 TensorFlow 中如何使用矩阵和各种数学操作。本章末尾讲述如何访问本书所需的数据源。

第 2 章介绍如何在计算图中连接第 1 章中的所有算法组件，创建一个简单的分类器。接着，介绍计算图、损失函数、反向传播和训练模型。

第 3 章重点讨论使用 TensorFlow 实现各种线性回归算法，比如，戴明回归、lasso 回归、岭回归、弹性网络回归和逻辑回归，也展示了如何在 TensorFlow 计算图中实现每种回归算法。

第 4 章介绍支持向量机（SVM）算法，展示如何在 TensorFlow 中实现线性 SVM 算法、非线性 SVM 算法和多分类 SVM 算法。

第 5 章展示如何使用数值度量、文本度量和归一化距离函数实现最近邻域法。我们使用最近邻域法进行地址间的记录匹配和 MNIST 数据库中手写数字的分类。

第 6 章讲述如何使用 TensorFlow 实现神经网络算法，包括操作门和激励函数的概念。随后展示一个简单的神经网络并讨论如何建立不同类型的神经网络层。本章末尾通过神经网络算法教 TensorFlow 玩井字棋游戏。

第 7 章阐述借助 TensorFlow 实现的各种文本处理算法。我们展示如何实现文本的“词袋”和 TF-IDF 算法。然后介绍 CBOW 和 skip-gram 模型的神经网络文本表示方式，并对于

Word2Vec 和 Doc2Vec 用这些方法来做预测。

第 8 章扩展神经网络算法，说明如何借助卷积神经网络（CNN）算法在图像上应用神经网络算法。我们展示如何构建一个简单的 CNN 进行 MNIST 数字识别，并扩展到 CIFAR-10 任务中的彩色图片，也阐述了如何针对自定义任务扩展之前训练的图像识别模型。本章末尾详细解释 TensorFlow 实现的模仿大师绘画和 Deep-Dream 算法。

第 9 章解释在 TensorFlow 中如何实现递归神经网络（RNN）算法，展示如何进行垃圾短信预测和在莎士比亚文本样本集上扩展 RNN 模型生成文本。接着训练 Seq2Seq 模型实现德语 - 英语的翻译。本章末尾展示如何用孪生 RNN 模型进行地址记录匹配。

第 10 章介绍 TensorFlow 产品级用例和开发提示，同时介绍如何利用多处理设备（比如，GPU）和在多个设备上实现分布式 TensorFlow。

第 11 章展示 TensorFlow 如何实现 k-means 算法、遗传算法和求解常微分方程（ODE），还介绍了 Tensorboard 的各种用法和如何查看计算图指标。

阅读本书前的准备

书中的章节都会使用 TensorFlow，其官网为 <https://www.tensorflow.org/>，它是基于 Python 3（<https://www.python.org/downloads/>）编写的。大部分章节需要访问从网络中下载的数据集。

本书的目标读者

本书适用于有经验的机器学习读者和 Python 程序员。有机器学习背景的读者会发现 TensorFlow 的代码很有启发性；有 Python 编程经验的读者会觉得代码注释极具参考性。

模块说明

在本书中，你会频繁看到开始、动手做、工作原理、延伸学习和参考这几个模块。

为了系统地学习相关技术，下面简单解释一下：

□ 开始

该节告诉读者该技术的内容，描述如何准备软件或者前期的准备工作。

□ 动手做

具体的操作步骤。

□ 工作原理

详细解释前一节发生了什么。

□ 延伸学习

附加资源，以供读者延伸学习。

□ 参考

提供有用的链接和有帮助的资源信息。

下载示例代码

读者可登录华章网站 (www.hzbook.com) 下载本书示例代码文件。

CONTENTS

目 录

译者序

作者简介

审校者简介

前言

第 1 章 TensorFlow 基础 1

1.1 TensorFlow 介绍 1

1.2 TensorFlow 如何工作 1

1.2.1 开始 1

1.2.2 动手做 2

1.2.3 工作原理 3

1.2.4 参考 3

1.3 声明张量 3

1.3.1 开始 4

1.3.2 动手做 4

1.3.3 工作原理 5

1.3.4 延伸学习 5

1.4 使用占位符和变量 6

1.4.1 开始 6

1.4.2 动手做 6

1.4.3 工作原理 6

1.4.4 延伸学习 7

1.5 操作 (计算) 矩阵 7

1.5.1 开始 7

1.5.2 动手做 8

1.5.3 工作原理 9

1.6 声明操作 10

1.6.1 开始 10

1.6.2 动手做 10

1.6.3 工作原理 11

1.6.4 延伸学习 12

1.7 实现激励函数 12

1.7.1 开始 12

1.7.2 动手做 12

1.7.3 工作原理 13

1.7.4 延伸学习 13

1.8 读取数据源 14

1.8.1 开始 15

1.8.2 动手做 15

1.8.3 参考 18

1.9 学习资料 19

第 2 章 TensorFlow 进阶 20

2.1 本章概要 20

2.2 计算图中的操作 20

2.2.1	开始	20
2.2.2	动手做	21
2.2.3	工作原理	21
2.3	TensorFlow 的嵌入 Layer	21
2.3.1	开始	21
2.3.2	动手做	22
2.3.3	工作原理	22
2.3.4	延伸学习	22
2.4	TensorFlow 的多层 Layer	23
2.4.1	开始	23
2.4.2	动手做	24
2.4.3	工作原理	25
2.5	TensorFlow 实现损失函数	26
2.5.1	开始	26
2.5.2	动手做	26
2.5.3	工作原理	28
2.5.4	延伸学习	29
2.6	TensorFlow 实现反向传播	30
2.6.1	开始	30
2.6.2	动手做	31
2.6.3	工作原理	33
2.6.4	延伸学习	34
2.6.5	参考	34
2.7	TensorFlow 实现随机训练和批量训练	34
2.7.1	开始	35
2.7.2	动手做	35
2.7.3	工作原理	36
2.7.4	延伸学习	37
2.8	TensorFlow 实现创建分类器	37
2.8.1	开始	37

2.8.2	动手做	37
2.8.3	工作原理	39
2.8.4	延伸学习	40
2.8.5	参考	40
2.9	TensorFlow 实现模型评估	40
2.9.1	开始	40
2.9.2	动手做	41
2.9.3	工作原理	41

第 3 章 基于 TensorFlow 的 线性回归

3.1	线性回归介绍	45
3.2	用 TensorFlow 求逆矩阵	45
3.2.1	开始	45
3.2.2	动手做	46
3.2.3	工作原理	47
3.3	用 TensorFlow 实现矩阵分解	47
3.3.1	开始	47
3.3.2	动手做	47
3.3.3	工作原理	48
3.4	用 TensorFlow 实现线性回归 算法	49
3.4.1	开始	49
3.4.2	动手做	49
3.4.3	工作原理	52
3.5	理解线性回归中的损失函数	52
3.5.1	开始	52
3.5.2	动手做	52
3.5.3	工作原理	53
3.5.4	延伸学习	54
3.6	用 TensorFlow 实现戴明回归 算法	55

3.6.1 开始	55
3.6.2 动手做	56
3.6.3 工作原理	57
3.7 用 TensorFlow 实现 lasso 回归和岭回归算法	58
3.7.1 开始	58
3.7.2 动手做	58
3.7.3 工作原理	59
3.7.4 延伸学习	59
3.8 用 TensorFlow 实现弹性网络回归算法	60
3.8.1 开始	60
3.8.2 动手做	60
3.8.3 工作原理	61
3.9 用 TensorFlow 实现逻辑回归算法	62
3.9.1 开始	62
3.9.2 动手做	62
3.9.3 工作原理	65

第 4 章 基于 TensorFlow 的支持

向量机	66
4.1 支持向量机简介	66
4.2 线性支持向量机的使用	67
4.2.1 开始	67
4.2.2 动手做	68
4.2.3 工作原理	72
4.3 弱化为线性回归	72
4.3.1 开始	73
4.3.2 动手做	73
4.3.3 工作原理	76
4.4 TensorFlow 上核函数的使用	77

4.4.1 开始	77
4.4.2 动手做	77
4.4.3 工作原理	81
4.4.4 延伸学习	82
4.5 用 TensorFlow 实现非线性支持向量机	82
4.5.1 开始	82
4.5.2 动手做	82
4.5.3 工作原理	84
4.6 用 TensorFlow 实现多类支持向量机	85
4.6.1 开始	85
4.6.2 动手做	86
4.6.3 工作原理	89

第 5 章 最近邻域法

5.1 最近邻域法介绍	90
5.2 最近邻域法的使用	91
5.2.1 开始	91
5.2.2 动手做	91
5.2.3 工作原理	94
5.2.4 延伸学习	94
5.3 如何度量文本距离	95
5.3.1 开始	95
5.3.2 动手做	95
5.3.3 工作原理	98
5.3.4 延伸学习	98
5.4 用 TensorFlow 实现混合距离计算	98
5.4.1 开始	98
5.4.2 动手做	98
5.4.3 工作原理	101

5.4.4	延伸学习	101	6.5.3	工作原理	126
5.5	用 TensorFlow 实现地址匹配	101	6.6	用 TensorFlow 实现多层神经网络	126
5.5.1	开始	101	6.6.1	开始	126
5.5.2	动手做	102	6.6.2	动手做	126
5.5.3	工作原理	104	6.6.3	工作原理	131
5.6	用 TensorFlow 实现图像识别	105	6.7	线性预测模型的优化	131
5.6.1	开始	105	6.7.1	开始	131
5.6.2	动手做	105	6.7.2	动手做	131
5.6.3	工作原理	108	6.7.3	工作原理	135
5.6.4	延伸学习	108	6.8	用 TensorFlow 基于神经网络实现井字棋	136
第 6 章	神经网络算法	109	6.8.1	开始	136
6.1	神经网络算法基础	109	6.8.2	动手做	137
6.2	用 TensorFlow 实现门函数	110	6.8.3	工作原理	142
6.2.1	开始	110	第 7 章	自然语言处理	143
6.2.2	动手做	111	7.1	文本处理介绍	143
6.2.3	工作原理	113	7.2	词袋的使用	144
6.3	使用门函数和激励函数	113	7.2.1	开始	144
6.3.1	开始	114	7.2.2	动手做	144
6.3.2	动手做	114	7.2.3	工作原理	149
6.3.3	工作原理	116	7.2.4	延伸学习	149
6.3.4	延伸学习	117	7.3	用 TensorFlow 实现 TF-IDF 算法	149
6.4	用 TensorFlow 实现单层神经网络	117	7.3.1	开始	150
6.4.1	开始	117	7.3.2	动手做	150
6.4.2	动手做	117	7.3.3	工作原理	154
6.4.3	工作原理	119	7.3.4	延伸学习	154
6.4.4	延伸学习	119	7.4	用 TensorFlow 实现 skip-gram 模型	155
6.5	用 TensorFlow 实现神经网络常见层	120	7.4.1	开始	155
6.5.1	开始	120			
6.5.2	动手做	121			

7.4.2 动手做	155	8.3.1 开始	188
7.4.3 工作原理	162	8.3.2 动手做	189
7.4.4 延伸学习	162	8.3.3 工作原理	196
7.5 用 TensorFlow 实现 CBOW 词 嵌入模型	162	8.3.4 参考	196
7.5.1 开始	162	8.4 再训练已有的 CNN 模型	196
7.5.2 动手做	163	8.4.1 开始	196
7.5.3 工作原理	167	8.4.2 动手做	196
7.5.4 延伸学习	167	8.4.3 工作原理	199
7.6 使用 TensorFlow 的 Word2Vec 预测	167	8.4.4 参考	199
7.6.1 开始	167	8.5 用 TensorFlow 实现模仿大师 绘画	199
7.6.2 动手做	168	8.5.1 开始	200
7.6.3 工作原理	172	8.5.2 动手做	200
7.6.4 延伸学习	172	8.5.3 工作原理	205
7.7 用 TensorFlow 实现基于 Doc2Vec 的情感分析	172	8.5.4 参考	205
7.7.1 开始	172	8.6 用 TensorFlow 实现 DeepDream	205
7.7.2 动手做	173	8.6.1 开始	205
7.7.3 工作原理	180	8.6.2 动手做	205
		8.6.3 延伸学习	210
		8.6.4 参考	210
第 8 章 卷积神经网络	181	第 9 章 递归神经网络	211
8.1 卷积神经网络介绍	181	9.1 递归神经网络介绍	211
8.2 用 TensorFlow 实现简单的 CNN	182	9.2 用 TensorFlow 实现 RNN 模型 进行垃圾短信预测	212
8.2.1 开始	182	9.2.1 开始	212
8.2.2 动手做	182	9.2.2 动手做	213
8.2.3 工作原理	187	9.2.3 工作原理	217
8.2.4 延伸学习	188	9.2.4 延伸学习	218
8.2.5 参考	188	9.3 用 TensorFlow 实现 LSTM 模型	218
8.3 用 TensorFlow 实现进阶的 CNN	188	9.3.1 开始	218
		9.3.2 动手做	219

9.3.3 工作原理	226	10.5 TensorFlow 产品化开发提示	252
9.3.4 延伸学习	226	10.5.1 开始	252
9.4 Stacking 多个 LSTM Layer	226	10.5.2 动手做	252
9.4.1 开始	226	10.5.3 工作原理	254
9.4.2 动手做	227	10.6 TensorFlow 产品化的实例	254
9.4.3 工作原理	228	10.6.1 开始	254
9.5 用 TensorFlow 实现 Seq2Seq 翻译模型	229	10.6.2 动手做	254
9.5.1 开始	229	10.6.3 工作原理	256
9.5.2 动手做	229		
9.5.3 工作原理	234	第 11 章 TensorFlow 的进阶应用	257
9.5.4 延伸学习	234	11.1 简介	257
9.6 TensorFlow 实现孪生 RNN 预测相似度	235	11.2 TensorFlow 可视化: Tensorboard	257
9.6.1 开始	235	11.2.1 开始	257
9.6.2 动手做	236	11.2.2 动手做	258
9.6.3 延伸学习	242	11.3 Tensorboard 的进阶	260
第 10 章 TensorFlow 产品化	243	11.4 用 TensorFlow 实现遗传算法	262
10.1 简介	243	11.4.1 开始	262
10.2 TensorFlow 的单元测试	243	11.4.2 动手做	263
10.2.1 开始	243	11.4.3 工作原理	265
10.2.2 工作原理	247	11.4.4 延伸学习	266
10.3 TensorFlow 的并发执行	247	11.5 TensorFlow 实现 k-means 算法	266
10.3.1 开始	248	11.5.1 开始	266
10.3.2 动手做	248	11.5.2 动手做	266
10.3.3 工作原理	250	11.5.3 延伸学习	270
10.3.4 延伸学习	250	11.6 用 TensorFlow 求解常微分方程问题	270
10.4 分布式 TensorFlow 实践	250	11.6.1 开始	270
10.4.1 开始	250	11.6.2 动手做	270
10.4.2 动手做	250	11.6.3 工作原理	271
10.4.3 工作原理	251	11.6.4 参考	272

第1章

TensorFlow 基础

本章将介绍 TensorFlow 的基本概念，帮助读者去理解 TensorFlow 是如何工作的，以及它如何访问数据集和学习资源。学完本章可以掌握以下知识点：

□ TensorFlow 如何工作

□ 声明变量和张量

□ 占位符和变量的用法

□ 矩阵的使用

□ 声明计算操作

□ 实现激励函数

□ 读取数据源

□ 学习资料

1.1 TensorFlow 介绍

Google 的 TensorFlow 引擎提供了一种解决机器学习问题的高效方法。机器学习在各行各业应用广泛，特别是计算机视觉、语音识别、语言翻译和健康医疗等领域。本书将详细介绍 TensorFlow 操作的基本步骤以及代码。这些基础知识对理解本书后续章节非常有用。

1.2 TensorFlow 如何工作

首先，TensorFlow 的计算看起来并不是很复杂，因为 TensorFlow 的计算过程和算法开发相当容易。这章将引导读者理解 TensorFlow 算法的伪代码。

1.2.1 开始

截至目前，TensorFlow 支持 Linux、Mac 和 Windows 操作系统。本书的代码都是在 Linux 操作系统上实现和运行的，不过运行在其他操作系统上也没问题。本书的代码可以在 GitHub

(https://github.com/nfmcclure/tensorflow_cookbookTensorFlow) 上获取。虽然 TensorFlow 是用 C++ 编写,但是全书只介绍 TensorFlow 的 Python 使用方式。本书将使用 Python 3.4+ (<https://www.python.org>) 和 TensorFlow 0.12 (<https://www.tensorflow.org>)。TensorFlow 官方已经在 GitHub 上发布 1.0.0 alpha 版本,本书代码兼容相应版本。TensorFlow 能在 CPU 上运行,大部分算法在 GPU 上会运行得更快,它支持英伟达显卡 (Nvidia Compute Capability v4.0+, 推荐 v5.1)。TensorFlow 上常用的 GPU 是英伟达特斯拉 (Nvidia Tesla) 和英伟达帕斯卡 (Nvidia Pascal),至少需要 4GB 的 RAM。为了运行 GPU,需要下载 Nvidia Cuda Toolkit 及其 v5.x 版本 (<https://developer.nvidia.com/cuda-downloads>)。本书还依赖 Python 的包: Scipy、Numpy 和 Scikit-Learn。

1.2.2 动手做

这里是 TensorFlow 算法的一般流程,本书提炼出的纲领如下:

1. 导入 / 生成样本数据集: 所有的机器学习算法都依赖样本数据集,本书的数据集既有生成的样本数据集,也有外部公开的样本数据集。有时,生成的数据集会更容易符合预期结果,但是本书大部分都是访问外部公开的样本数据集,具体细节见第 8 章。

2. 转换和归一化数据: 一般来讲,输入样本数据集并不符合 TensorFlow 期望的形状,所以需要转换数据格式以满足 TensorFlow。当数据集的维度或者类型不符合所用机器学习算法的要求时,需要在使用前进行数据转换。大部分机器学习算法期待的输入样本数据是归一化的数据。TensorFlow 具有内建函数来归一化数据,如下:

```
data = tf.nn.batch_norm_with_global_normalization(...)
```

3. 划分样本数据集为训练样本集、测试样本集和验证样本集: 一般要求机器学习算法的训练样本集和测试样本集是不同的数据集。另外,许多机器学习算法要求超参数调优,所以需要验证样本集来决定最优的超参数。

4. 设置机器学习参数 (超参数): 机器学习经常要有一系列的常量参数。例如,迭代次数、学习率,或者其他固定参数。约定俗成的习惯是一次性初始化所有的机器学习参数,读者经常看到的形式如下:

```
learning_rate = 0.01
batch_size = 100
iterations = 1000
```

5. 初始化变量和占位符: 在求解最优化过程中 (最小化损失函数),TensorFlow 通过占位符获取数据,并调整变量和权重 / 偏差。TensorFlow 指定数据大小和数据类型初始化变量和占位符。本书大部分使用 float32 数据类型,TensorFlow 也支持 float64 和 float16。注意,使用的数据类型字节数越多结果越精确,同时运行速度越慢。使用方式如下:

```
a_var = tf.constant(42)
x_input = tf.placeholder(tf.float32, [None, input_size])
y_input = tf.placeholder(tf.float32, [None, num_classes])
```

6. 定义模型结构：在获取样本数据集、初始化变量和占位符后，开始定义机器学习模型。TensorFlow 通过选择操作、变量和占位符的值来构建计算图，详细讲解见第2章。这里给出简单的线性模型：

```
y_pred = tf.add(tf.mul(x_input, weight_matrix), b_matrix)
```

7. 声明损失函数：定义完模型后，需要声明损失函数来评估输出结果。损失函数能说明预测值与实际值的差距，损失函数的种类将在第2章详细展示：

```
loss = tf.reduce_mean(tf.square(y_actual - y_pred))
```

8. 初始化模型和训练模型：TensorFlow 创建计算图实例，通过占位符赋值，维护变量的状态信息。下面是初始化计算图的一种方式：

```
with tf.Session(graph=graph) as session:
```

```
...
    session.run(...)
...

```

也可以用如下方式初始化计算图：

```
session = tf.Session(graph=graph)
session.run(...)
```

9. 评估机器学习模型：一旦构建计算图，并训练机器学习模型后，需要寻找某种标准来评估机器学习模型对新样本数据集的效果。通过对训练样本集和测试样本集的评估，可以确定机器学习模型是过拟合还是欠拟合。这些将在后续章节来解决。

10. 调优超参数：大部分情况下，机器学习者需要基于模型效果来回调整一些超参数。通过调整不同的超参数来重复训练模型，并用验证样本集来评估机器学习模型。

11. 发布 / 预测结果：所有机器学习模型一旦训练好，最后都用来预测新的、未知的数据。

1.2.3 工作原理

使用 TensorFlow 时，必须准备样本数据集、变量、占位符和机器学习模型，然后进行模型训练，改变变量状态来提高预测结果。TensorFlow 通过计算图实现上述过程。这些计算图是有向无环图，并且支持并行计算。接着 TensorFlow 创建损失函数，通过调整计算图中的变量来最小化损失函数。TensorFlow 维护模型的计算状态，每步迭代自动计算梯度。

1.2.4 参考

□ https://www.tensorflow.org/api_docs/python/

□ <https://www.tensorflow.org/tutorials/>

1.3 声明张量

TensorFlow 的主要数据结构是张量，它用张量来操作计算图。在 TensorFlow 里可以把

变量或者占位符声明为张量。首先，需要知道如何创建张量。

1.3.1 开始

创建一个张量，声明其为一个变量。TensorFlow 在计算图中可以创建多个图结构。这里需要指出，在 TensorFlow 中创建一个张量，并不会立即在计算图中增加什么。只有把张量赋值给一个变量或者占位符，TensorFlow 才会把此张量增加到计算图。更多信息请见下一章。

1.3.2 动手做

这里将介绍在 TensorFlow 中创建张量的主要方法：

1. 固定张量：

❑ 创建指定维度的零张量。使用方式如下：

```
zero_tsr = tf.zeros([row_dim, col_dim])
```

❑ 创建指定维度的单位张量。使用方式如下：

```
ones_tsr = tf.ones([row_dim, col_dim])
```

❑ 创建指定维度的常数填充的张量。使用方式如下：

```
filled_tsr = tf.fill([row_dim, col_dim], 42)
```

❑ 用已知常数张量创建一个张量。使用方式如下：

```
constant_tsr = tf.constant([1,2,3])
```



tf.constant() 函数也能广播一个值为数组，然后模拟 tf.fill() 函数的功能，具体写法为：tf.constant(42, [row_dim, col_dim])。

2. 相似形状的张量：

❑ 新建一个与给定的 tensor 类型大小一致的 tensor，其所有元素为 0 或者 1，使用方式如下：

```
zeros_similar = tf.zeros_like(constant_tsr)
```

```
ones_similar = tf.ones_like(constant_tsr)
```



因为这些张量依赖给定的张量，所以初始化时需要按序进行。如果打算一次性初始化所有张量，那么程序将会报错。

3. 序列张量：

❑ TensorFlow 可以创建指定间隔的张量。下面的函数的输出跟 range() 函数和 numpy

的 `linspace()` 函数的输出相似:

```
linear_tsr = tf.linspace(start=0, stop=1, steps=3)
```

- 返回的张量是 `[0.0, 0.5, 1.0]` 序列。注意，上面的函数结果中最后一个值是 `stop` 值。另外一个 `rang()` 函数的使用方式如下:

```
integer_seq_tsr = tf.range(start=6, limit=15, delta=3)
```

- 返回的张量是 `[6, 9, 12]`。注意，这个函数结果不包括 `limit` 值。

4. 随机张量:

- 下面的 `tf.random_uniform()` 函数生成均匀分布的随机数:

```
randunif_tsr = tf.random_uniform([row_dim, col_dim],
                                  minval=0, maxval=1)
```

- 注意，这个随机均匀分布从 `minval` (包含 `minval` 值) 开始到 `maxval` (不包含 `maxval` 值) 结束，即 $(\text{minval} \leq x < \text{maxval})$ 。

- `tf.random_normal()` 函数生成正态分布的随机数:

```
randnorm_tsr = tf.random_normal([row_dim, col_dim],
                                  mean=0.0, stddev=1.0)
```

- `tf.truncated_normal()` 函数生成带有指定边界的正态分布的随机数，其正态分布的随机数位于指定均值 (期望) 到两个标准差之间的区间:

```
runcnorm_tsr = tf.truncated_normal([row_dim, col_dim],
                                     mean=0.0, stddev=1.0)
```

- 张量 / 数组的随机化。`tf.random_shuffle()` 和 `tf.random_crop()` 可以实现此功能:

```
shuffled_output = tf.random_shuffle(input_tensor)
cropped_output = tf.random_crop(input_tensor, crop_size)
```

- 张量的随机剪裁。`tf.random_crop()` 可以实现对张量指定大小的随机剪裁。在本书的后面部分，会对具有 3 通道颜色的图像 (`height, width, 3`) 进行随机剪裁。为了固定剪裁结果的一个维度，需要在相应的维度上赋其最大值:

```
cropped_image = tf.random_crop(my_image, [height/2, width/2,
3])
```

1.3.3 工作原理

一旦创建好张量，就可以通过 `tf.Variable()` 函数封装张量来作为变量，更多细节见下节，使用方式如下:

```
my_var = tf.Variable(tf.zeros([row_dim, col_dim]))
```

1.3.4 延伸学习

创建张量并不一定得用 TensorFlow 内建函数，可以使用 `tf.convert_to_tensor()` 函数将

任意 numpy 数组转换为 Python 列表，或者将常量转换为一个张量。注意，`tf.convert_to_tensor()` 函数也可以接受张量作为输入。

1.4 使用占位符和变量

使用 TensorFlow 计算图的关键工具是占位符和变量，也请读者务必理解两者的区别，以及什么地方该用谁。

1.4.1 开始

使用数据的关键点之一是搞清楚它是占位符还是变量。变量是 TensorFlow 机器学习算法的参数，TensorFlow 维护（调整）这些变量的状态来优化机器学习算法。占位符是 TensorFlow 对象，用于表示输入输出数据的格式，允许传入指定类型和形状的数据，并依赖计算图的计算结果，比如，期望的计算结果。

1.4.2 动手做

在 TensorFlow 中，`tf.Variable()` 函数创建变量，过程是输入一个张量，返回一个变量。声明变量后需要初始化变量。下面是创建变量并初始化的例子：

```
my_var = tf.Variable(tf.zeros([2,3]))
sess = tf.Session()
initialize_op = tf.global_variables_initializer()
sess.run(initialize_op)
```

占位符仅仅声明数据位置，用于传入数据到计算图。占位符通过会话的 `feed_dict` 参数获取数据。在计算图中使用占位符时，必须在其上执行至少一个操作。在 TensorFlow 中，初始化计算图，声明一个占位符 `x`，定义 `y` 为 `x` 的 identity 操作。identity 操作返回占位符传入的数据本身。结果图将在下节展示，代码如下：

```
sess = tf.Session()
x = tf.placeholder(tf.float32, shape=[2,2])
y = tf.identity(x)
x_vals = np.random.rand(2,2)
sess.run(y, feed_dict={x: x_vals})
# Note that sess.run(x, feed_dict={x: x_vals}) will result in a self-
referencing error.
```

1.4.3 工作原理

以零张量初始化变量，其计算图如图 1-1 所示。

在图 1-1 中可以看出，计算图仅仅有一个变量，全部初始化为 0。图中灰色部分详细地展示计算图操作以及相关的常量。右上角的小图展示的是主计算图。关于在 TensorFlow 中创建和可视化计算图的部分见第 10 章。

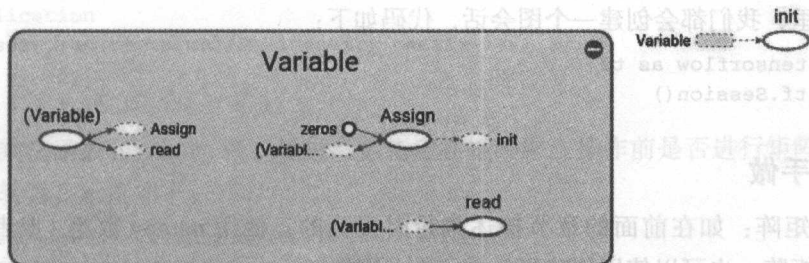


图 1-1 变量

相似地，一个占位符传入 numpy 数组的计算图展示如图 1-2 所示。

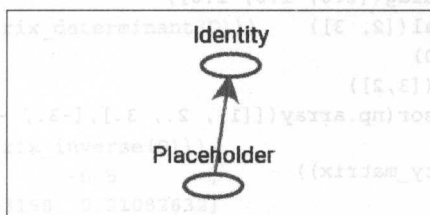


图 1-2 占位符初始化的计算图。灰色部分详细地展示计算图操作以及相关的常量

1.4.4 延伸学习

在计算图运行的过程中，需要告诉 TensorFlow 初始化所创建的变量。TensorFlow 的每个变量都有 `initializer` 方法，但最常用的方式是 helper 函数 (`global_variables_initializer()`)。此函数会一次性初始化所创建的所有变量，使用方式如下：

```
initializer_op = tf.global_variables_initializer()
```

但是，如果是基于已经初始化的变量进行初始化，则必须按序进行初始化，使用方式如下：

```
sess = tf.Session()
first_var = tf.Variable(tf.zeros([2,3]))
sess.run(first_var.initializer)
second_var = tf.Variable(tf.zeros_like(first_var))
# Depends on first_var
sess.run(second_var.initializer)
```

1.5 操作 (计算) 矩阵

理解 TensorFlow 如何操作矩阵，对于理解计算图中数据的流动来说非常重要。

1.5.1 开始

许多机器学习算法依赖矩阵操作。在 TensorFlow 中，矩阵计算是相当容易的。在下面

的所有例子里，我们都会创建一个图会话，代码如下：

```
import tensorflow as tf
sess = tf.Session()
```

1.5.2 动手做

1. 创建矩阵：如在前面的章节描述张量时提到的，使用 `numpy` 数组（或者嵌套列表）来创建二维矩阵。也可以使用创建张量的函数（比如，`zeros()`、`ones()`、`truncated_normal()` 等），并为其指定一个二维形状。TensorFlow 也可以使用 `diag()` 函数从一个一维数组（或者列表）来创建对角矩阵，代码如下：

```
identity_matrix = tf.diag([1.0, 1.0, 1.0])
A = tf.truncated_normal([2, 3])
B = tf.fill([2,3], 5.0)
C = tf.random_uniform([3,2])
D = tf.convert_to_tensor(np.array([[1., 2., 3.],[-3., -7.,
-1.],[0., 5., -2.])))
print(sess.run(identity_matrix))
[[ 1.  0.  0.]
 [ 0.  1.  0.]
 [ 0.  0.  1.]]
print(sess.run(A))
[[ 0.96751703  0.11397751 -0.3438891 ]
 [-0.10132604 -0.8432678  0.29810596]]
print(sess.run(B))
[[ 5.  5.  5.]
 [ 5.  5.  5.]]
print(sess.run(C))
[[ 0.33184157  0.08907614]
 [ 0.53189191  0.67605299]
 [ 0.95889051  0.67061249]]
print(sess.run(D))
[[ 1.  2.  3.]
 [-3. -7. -1.]
 [ 0.  5. -2.]]
```



注意，如果再次运行 `sess.run(C)`，TensorFlow 会重新初始化随机变量，并得到不同的随机数。

2. 矩阵的加法和减法：

```
print(sess.run(A+B))
[[ 4.61596632  5.39771316  4.4325695 ]
 [ 3.26702736  5.14477345  4.98265553]]
print(sess.run(B-B))
[[ 0.  0.  0.]
 [ 0.  0.  0.]]
```

```
Multiplication
print(sess.run(tf.matmul(B, identity_matrix)))
[[ 5.  5.  5.]
 [ 5.  5.  5.]]
```

3. 矩阵乘法函数 `matmul()` 可以通过参数指定在矩阵乘法操作前是否进行矩阵转置。

4. 矩阵转置，示例如下：

```
print(sess.run(tf.transpose(C)))
[[ 0.67124544  0.26766731  0.99068872]
 [ 0.25006068  0.86560275  0.58411312]]
```

5. 再次强调，重新初始化将会得到不同的值。

6. 对于矩阵行列式，使用方式如下：

```
print(sess.run(tf.matrix_determinant(D)))
-38.0
```

❑ 矩阵的逆矩阵：

```
print(sess.run(tf.matrix_inverse(D)))
[[-0.5         -0.5         -0.5        ]
 [ 0.15789474  0.05263158  0.21052632]
 [ 0.39473684  0.13157895  0.02631579]]
```



TensorFlow 中的矩阵求逆方法是 Cholesky 矩阵分解法（又称为平方根法），矩阵需要为对称正定矩阵或者可进行 LU 分解。

7. 矩阵分解：

❑ Cholesky 矩阵分解法，使用方式如下：

```
print(sess.run(tf.cholesky(identity_matrix)))
[[ 1.  0.  1.]
 [ 0.  1.  0.]
 [ 0.  0.  1.]]
```

8. 矩阵的特征值和特征向量，使用方式如下：

```
print(sess.run(tf.self_adjoint_eig(D)))
[[-10.65907521 -0.22750691  2.88658212]
 [ 0.21749542  0.63250104 -0.74339638]
 [ 0.84526515  0.2587998  0.46749277]
 [-0.4880805  0.73004459  0.47834331]]
```

注意，`self_adjoint_eig()` 函数的输出结果中，第一行为特征值，剩下的向量是对应的向量。在数学中，这种方法也称为矩阵的特征分解。

1.5.3 工作原理

TensorFlow 提供数值计算工具，并把这些计算添加到计算图中。这些部分对于简单的

矩阵计算来说看似有点重，TensorFlow 增加这些矩阵操作到计算图进行张量计算。现在看起来这些介绍有些啰嗦，但是这有助于理解后续章节的内容。

1.6 声明操作

现在开始学习 TensorFlow 计算图的其他操作。

1.6.1 开始

除了标准数值计算外，TensorFlow 提供很多其他的操作。在使用之前，按照惯例创建一个计算图会话，代码如下：

```
import tensorflow as tf
sess = tf.Session()
```

1.6.2 动手做

TensorFlow 张量的基本操作有：add()、sub()、mul() 和 div()。注意，除特别说明外，这章节所有的操作都是对张量的每个元素进行操作：

1. TensorFlow 提供 div() 函数的多种变种形式和相关的函数。

2. 值得注意的，div() 函数返回值的数据类型与输入数据类型一致。这意味着，在 Python 2 中，整数除法的实际返回是商的向下取整，即不大于商的最大整数；而 Python 3 版本中，TensorFlow 提供 truediv() 函数，其会在除法操作前强制转换整数为浮点数，所以最终的除法结果是浮点数，代码如下：

```
print(sess.run(tf.div(3,4)))
0
print(sess.run(tf.truediv(3,4)))
0.75
```

3. 如果要对浮点数进行整数除法，可以使用 floordiv() 函数。注意，此函数也返回浮点数结果，但是其会向下舍去小数位到最近的整数。示例如下：

```
print(sess.run(tf.floordiv(3.0,4.0)))
0.0
```

4. 另外一个重要的函数是 mod()（取模）。此函数返回除法的余数。示例如下：

```
print(sess.run(tf.mod(22.0, 5.0)))
2.0
```

5. 通过 cross() 函数计算两个张量间的点积。记住，点积函数只为三维向量定义，所以 cross() 函数以两个三维张量作为输入，示例如下：

```
print(sess.run(tf.cross([1., 0., 0.], [0., 1., 0.])))
[ 0.  0.  1.0]
```

6. 下面给出数学函数的列表:

abs()	返回输入参数张量的绝对值
ceil()	返回输入参数张量的向上取整结果
cos()	返回输入参数张量的余弦值
exp()	返回输入参数张量的自然常数 e 的指数
floor()	返回输入参数张量的向下取整结果
inv()	返回输入参数张量的倒数
log()	返回输入参数张量的自然对数
maximum()	返回两个输入参数张量中的最大值
minimum()	返回两个输入参数张量中的最小值
neg()	返回输入参数张量的负值
pow()	返回输入参数第一个张量的第二个张量的次幂
round()	返回输入参数张量的四舍五入结果
rsqrt()	返回输入参数张量的平方根的倒数
sign()	根据输入参数张量的符号, 返回 -1、0 或 1
sin()	返回输入参数张量的正弦值
sqrtd()	返回输入参数张量的平方根
square()	返回输入参数张量的平方

7. 特殊数学函数: 有些用在机器学习中的特殊数学函数值得一提, TensorFlow 也有对应的内建函数。除特别说明外, 这些函数操作的也是张量的每个元素。

digamma()	普西函数 (Psi 函数), lgamma() 函数的导数
erf()	返回张量的高斯误差函数
erfc()	返回张量的互补误差函数
igamma()	返回下不完全伽马函数
igammac()	返回上不完全伽马函数
lbeta()	返回贝塔函数绝对值的自然对数
lgamma()	返回伽马函数绝对值的自然对数
squared_difference()	返回两个张量间差值的平方

1.6.3 工作原理

知道在计算图中应用什么函数合适是重要的。大部分情况下, 我们关心预处理函数, 但也通过组合预处理函数生成许多自定义函数, 示例如下:

```
# Tangent function (tan(pi/4)=1)
print(sess.run(tf.div(tf.sin(3.1416/4.), tf.cos(3.1416/4.))))
```

1.0

1.6.4 延伸学习

如果希望（未在上述函数列表中列出的操作）在计算图中增加其他操作，必须创建自定义函数。下面创建一个自定义二次多项式函数， $3x^2 - x + 10$ ：

```
def custom_polynomial(value):
    return(tf.sub(3 * tf.square(value), value) + 10)
print(sess.run(custom_polynomial(11)))
362
```

1.7 实现激励函数

1.7.1 开始

激励函数是使用所有神经网络算法的必备“神器”。激励函数的目的是为了调节权重和误差。在 TensorFlow 中，激励函数是作用在张量上的非线性操作。激励函数的使用方法和前面的数学操作相似。激励函数的功能有很多，但其主要是为计算图归一化返回结果而引进的非线性部分。创建一个 TensorFlow 计算图：

```
import tensorflow as tf
sess = tf.Session()
```

1.7.2 动手做

TensorFlow 的激励函数位于神经网络（neural network，nn）库。除了使用 TensorFlow 内建激励函数外，我们也可以使用 TensorFlow 操作设计自定义激励函数。导入预定义激励函数，或者在函数中显式调用 .nn。这里，选择每个函数显式调用的方法。

1. 整流线性单元（Rectifier linear unit，ReLU）是神经网络最常用的非线性函数。其函数为 $\max(0, x)$ ，连续但不平滑。示例如下：

```
print(sess.run(tf.nn.relu([-3., 3., 10.])))
[ 0.  3. 10.]
```

2. 有时为了抵消 ReLU 激励函数的线性增长部分，会在 min() 函数中嵌入 $\max(0, x)$ ，其在 TensorFlow 中的实现称作 ReLU6，表示为 $\min(\max(0, x), 6)$ 。这是 hard-sigmoid 函数的变种，计算运行速度快，解决梯度消失（无限趋近于 0），这些将在第 8 章和第 9 章中详细阐述，使用方式如下：

```
print(sess.run(tf.nn.relu6([-3., 3., 10.])))
[ 0.  3.  6.]
```

3. sigmoid 函数是最常用的连续、平滑的激励函数。它也被称作逻辑函数（Logistic 函数），表示为 $1/(1+\exp(-x))$ 。sigmoid 函数由于在机器学习训练过程中反向传播项趋近于 0，因此不怎么使用。使用方式如下：

```
print(sess.run(tf.nn.sigmoid([-1., 0., 1.])))
[ 0.26894143  0.5          0.7310586 ]
```



注意，有些激励函数不以 0 为中心，比如，sigmoid 函数。在大部分计算图算法中要求优先使用均值为 0 的样本数据集。

4. 另外一种激励函数是双曲正切函数 (hyper tangent, tanh)。双曲正切函数与 sigmoid 函数相似，但有一点不同：双曲正切函数取值范围为 -1 到 1；sigmoid 函数取值范围为 -1 到 1。双曲正切函数是双曲正弦与双曲余弦的比值，另外一种写法是 $((\exp(x) - \exp(-x)) / (\exp(x) + \exp(-x)))$ 。使用方式如下：

```
print(sess.run(tf.nn.tanh([-1., 0., 1.])))
[-0.76159418  0.          0.76159418 ]
```

5. softsign 函数也是一种激励函数，表达式为： $x / (\text{abs}(x) + 1)$ 。softsign 函数是符号函数的连续估计，使用方式如下：

```
print(sess.run(tf.nn.softsign([-1., 0., -1.])))
[-0.5  0.   0.5]
```

6. softplus 激励函数是 ReLU 激励函数的平滑版，表达式为： $\log(\exp(x) + 1)$ 。使用方式如下：

```
print(sess.run(tf.nn.softplus([-1., 0., -1.])))
[ 0.31326166  0.69314718  1.31326163]
```



注意，当输入增加时，softplus 激励函数趋近于无限大，softsign 函数趋近于 1；当输入减小时，softplus 激励函数趋近于 0，softsign 函数趋近于 -1。

7. ELU 激励函数 (Exponential Linear Unit, ELU) 与 softplus 激励函数相似，不同点在于：当输入无限小时，ELU 激励函数趋近于 -1，而 softplus 激励函数趋近于 0。表达式为 $(\exp(x)+1)$ if $x < 0$ else x ，使用方式如下：

```
print(sess.run(tf.nn.elu([-1., 0., -1.])))
[-0.63212055  0.          1.          ]
```

1.7.3 工作原理

上面这些激励函数是神经网络引入的非线性部分，并需要知道在什么位置使用激励函数。如果激励函数的取值范围在 0 和 1 之间，比如 sigmoid 激励函数，那计算图输出结果也只能在 0 到 1 之间取值。

如果激励函数隐藏在节点之间，就要意识到激励函数作用于传入的张量的影响。如果张量要缩放为均值为 0，就需要使用激励函数使得尽可能多的变量在 0 附近。这暗示我们选用双曲正切 (tanh) 函数或者 softsign 函数。

1.7.4 延伸学习

图 1-3 和图 1-4 展示了不同激励函数，从中可以看到的激励函数有 ReLU、ReLU6、

softplus、ELU、sigmoid、softsign 和 tanh。

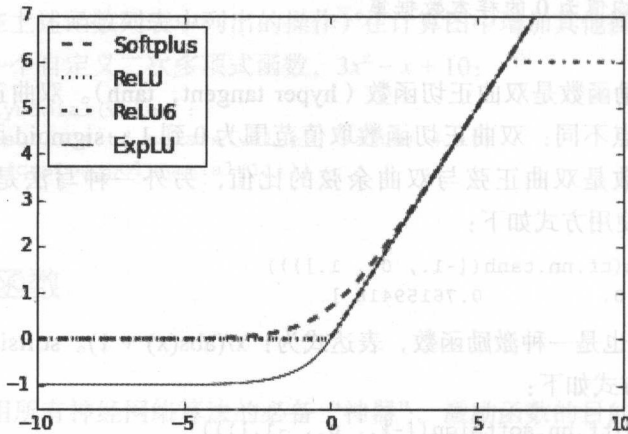


图 1-3 ReLU、ReLU6、softplus 和 ELU 激励函数

在图 1-3 中，我们可以看到四种激励函数：ReLU、ReLU6、softplus 和 ELU。这些激励函数输入值小于 0 时输出值逐渐变平，输入值大于 0 时输出值线性增长（除了 ReLU6 函数有最大值 6）。

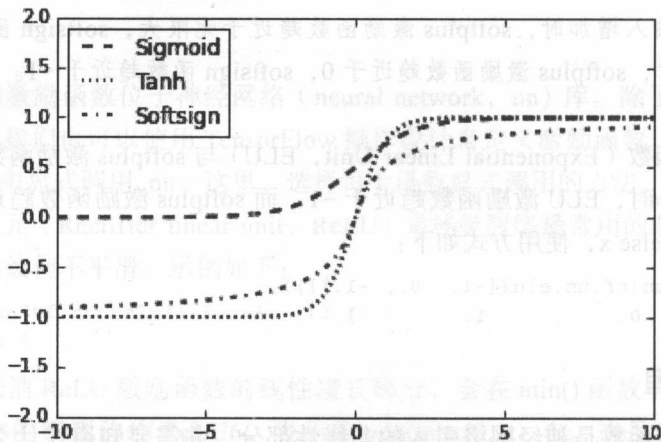


图 1-4 sigmoid、softsign 和 tanh 激励函数

图 1-4 展示的是 sigmoid 激励函数、双曲正切 (tanh) 激励函数和 softsign 激励函数。这些激励函数都是平滑的、具有 S 型，注意有两个激励函数有水平渐近线。

1.8 读取数据源

本书中使用样本数据集训练机器学习算法模型，本章简要介绍如何通过 TensorFlow 和 Python 访问各种数据源。

1.8.1 开始

在 TensorFlow 中，有些数据源使用 Python 内建库，有的需要编写 Python 脚本下载，还有其他的得手动从网上下载。所有这些数据源都需要联网才能获取到。

1.8.2 动手做

1. 鸢尾花卉数据集 (Iris data)。此样本数据是机器学习和统计分析最经典的数据集，包含山鸢尾、变色鸢尾和维吉尼亚鸢尾各自的花萼和花瓣的长度和宽度。总共有 150 个数据集，每类有 50 个样本。用 Python 加载样本数据集时，可以使用 Scikit Learn 的数据集函数，使用方式如下：

```
from sklearn import datasets
iris = datasets.load_iris()
print(len(iris.data))
150
print(len(iris.target))
150
print(iris.target[0]) # Sepal length, Sepal width, Petal length,
Petal width
[ 5.1 3.5 1.4 0.2]
print(set(iris.target)) # I. setosa, I. virginica, I. versicolor
{0, 1, 2}
```

2. 出生体重数据 (Birth weight data)。此样本数据集是婴儿出生体重以及母亲和家庭历史人口统计学、医学指标，有 189 个样本集，包含 11 个特征变量。使用 Python 访问数据的方式：

```
import requests
birthdata_url = 'https://www.umass.edu/statdata/statdata/data/
lowbwt.dat'
birth_file = requests.get(birthdata_url)
birth_data = birth_file.text.split('\r\n') [5:]
birth_header = [x for x in birth_data[0].split(' ') if len(x)>=1]
birth_data = [[float(x) for x in y.split(' ') if len(x)>=1] for y
in birth_data[1:] if len(y)>=1]
print(len(birth_data))
189
print(len(birth_data[0]))
11
```

3. 波士顿房价数据 (Boston Housing data)。此样本数据集保存在卡内基梅隆大学机器学习仓库，总共有 506 个房价样本，包含 14 个特征变量。使用 Python 获取数据的方式：

```
import requests
housing_url = 'https://archive.ics.uci.edu/ml/machine-learning-
databases/housing/housing.data'
housing_header = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM',
'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT', 'MEDV']
housing_file = requests.get(housing_url)
```

```
housing_data = [[float(x) for x in y.split(' ') if len(x)>=1] for
y in housing_file.text.split('\n') if len(y)>=1]
print(len(housing_data))
506
print(len(housing_data[0]))
14
```

4. MNIST 手写体字库: MNIST 手写体字库是 NIST 手写体字库的子样本数据集, 网址: <https://yann.lecun.com/exdb/mnist/>。包含 70 000 张 0 到 9 的图像, 其中 60 000 张标注为训练样本数据集, 10 000 张为测试样本数据集。TensorFlow 提供内建函数来访问它, MNIST 手写体字库常用来进行图像识别训练。在机器学习中, 提供验证样本数据集来预防过拟合是非常重要的, TensorFlow 从训练样本数据集中留出 5000 张图片作为验证样本数据集。这里展示使用 Python 访问数据的方式:

```
from tensorflow.examples.tutorials.mnist import input_data
mnist = input_data.read_data_sets("MNIST_data/", "one_hot=True")
print(len(mnist.train.images))
55000
print(len(mnist.test.images))
10000
print(len(mnist.validation.images))
5000
print(mnist.train.labels[1,:]) # The first label is a 3'''
[ 0.  0.  0.  1.  0.  0.  0.  0.  0.  0.]
```

5. 垃圾短信文本数据集 (Spam-ham text data)。通过以下方式访问垃圾短信文本数据:

```
import requests
import io
from zipfile import ZipFile
zip_url = 'http://archive.ics.uci.edu/ml/machine-learning-
databases/00228/smsspamcollection.zip'
r = requests.get(zip_url)
z = ZipFile(io.BytesIO(r.content))
file = z.read('SMSSpamCollection')
text_data = file.decode()
text_data = text_data.encode('ascii', errors='ignore')
text_data = text_data.decode().split('\n')
text_data = [x.split('\t') for x in text_data if len(x)>=1]
[text_data_target, text_data_train] = [list(x) for x in zip(*text_
data)]
print(len(text_data_train))
5574
print(set(text_data_target))
{'ham', 'spam'}
print(text_data_train[1])
Ok lar... Joking wif u oni...
```

6. 影评样本数据集。此样本数据集是电影观看者的影评, 分为好评和差评, 可以在网站 <http://www.cs.cornell.edu/people/pabo/movie-review-data/> 下载。这里用 Python 进行数据

处理, 使用方式如下:

```
import requests
import io
import tarfile
movie_data_url = 'http://www.cs.cornell.edu/people/pabo/movie-
review-data/rt-polaritydata.tar.gz'
r = requests.get(movie_data_url)
# Stream data into temp object
stream_data = io.BytesIO(r.content)
tmp = io.BytesIO()
while True:
    s = stream_data.read(16384)
    if not s:
        break
    tmp.write(s)
stream_data.close()
tmp.seek(0)
# Extract tar file
tar_file = tarfile.open(fileobj=tmp, mode="r:gz")
pos = tar_file.extractfile('rt'-polaritydata/rt-polarity.pos')
neg = tar_file.extractfile('rt'-polaritydata/rt-polarity.neg')
# Save pos/neg reviews (Also deal with encoding)
pos_data = []
for line in pos:
    pos_data.append(line.decode('ISO'-8859-1').
    encode('ascii', errors='ignore').decode())
neg_data = []
for line in neg:
    neg_data.append(line.decode('ISO'-8859-1').
    encode('ascii', errors='ignore').decode())
tar_file.close()
print(len(pos_data))
5331
print(len(neg_data))
5331
# Print out first negative review
print(neg_data[0])
simplistic , silly and tedious .
```

7. CIFAR-10 图像数据集。此图像数据集是 CIFAR 机构发布的 8 亿张彩色图片 (已标注为, 32×32 像素) 的子集, 总共分 10 类, 60 000 张图片。50 000 张图片训练数据集, 10 000 张测试数据集。由于这个图像数据集数据量大, 并在本书中以多种方式使用, 后面到具体用时再细讲, 访问网址为: <http://www.cs.toronto.edu/~kriz/cifar.html>。

8. 莎士比亚著作文本数据集 (Shakespeare text data)。此文本数据集是古登堡数字电子书计划提供的免费电子书籍, 他们编译了莎士比亚所有著作。用 Python 访问文本文件的方式如下:

```
import requests
shakespeare_url = 'http://www.gutenberg.org/cache/epub/100/pg100.
txt'
```



```
# Get Shakespeare text
response = requests.get(shakespeare_url)
shakespeare_file = response.content
# Decode binary into string
shakespeare_text = shakespeare_file.decode('utf-8')
# Drop first few descriptive paragraphs.
shakespeare_text = shakespeare_text[7675:]
print(len(shakespeare_text)) # Number of characters
5582212
```

9. 英德句子翻译样本集。此数据集由 Tatoeba（在线翻译数据库）发布，ManyThings.org（<http://www.manythings.org>）整理并提供下载。这里提供英德语句互译的文本文件（你可以通过改变 URL，使用你需要的任何语言的文本文件），使用方式如下：

```
import requests
import io
from zipfile import ZipFile
sentence_url = 'http://www.manythings.org/anki/deu-eng.zip'
r = requests.get(sentence_url)
z = ZipFile(io.BytesIO(r.content))
file = z.read('deu.txt')
# Format Data
eng_ger_data = file.decode()
eng_ger_data = eng_ger_data.encode('ascii', errors='ignore')
eng_ger_data = eng_ger_data.decode().split('\n')
eng_ger_data = [x.split('\t') for x in eng_ger_data if len(x)>=1]
[english_sentence, german_sentence] = [list(x) for x in zip(*eng_ger_data)]
print(len(english_sentence))
137673
print(len(german_sentence))
137673
print(eng_ger_data[10])
['I won!', 'Ich habe gewonnen!']
```

1.8.3 参考

- ▶ Hosmer, D.W., Lemeshow, S., and Sturdivant, R. X. (2013). Applied Logistic Regression: 3rd Edition. <https://www.umass.edu/statdata/statdata/data/lowbwt.txt>
- ▶ Lichman, M. (2013). UCI Machine Learning Repository. <http://archive.ics.uci.edu/ml>. Irvine, CA: University of California, School of Information and Computer Science.
- ▶ Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan, Thumbs up? Sentiment Classification using Machine Learning Techniques, Proceedings of EMNLP 2002. <http://www.cs.cornell.edu/people/pabo/movie-review-data/>
- ▶ Krizhevsky. (2009). Learning Multiple Layers of Features from Tiny Images. <http://www.cs.toronto.edu/~kriz/cifar.html>
- ▶ Project Gutenberg. Accessed April 2016. <http://www.gutenberg.org/>.

1.9 学习资料

这里提供一些关于 TensorFlow 使用和学习的链接、文档资料和用例。

TensorFlow 资源列表如下：

1. 本书代码可在 GitHub 获取 https://github.com/nfmcclure/tensorflow_cookbook。
2. TensorFlow 官方 Python API 文档地址：https://www.tensorflow.org/api_docs/python。其中包括 TensorFlow 所有函数、对象和方法的文档和例子。本书当前的 TensorFlow 版本为 r0.8。
3. TensorFlow 官方用例相当详细，访问网址：<https://www.tensorflow.org/tutorials/index.html>。包括图像识别模型、Word2Vec、RNN 模型和 sequence-to-sequence 模型，也有些偏微分方程的例子。后续还会不断增加更多实例。
4. TensorFlow 官方 GitHub 仓库：<https://github.com/tensorflow/tensorflow>。你可以查看源代码，甚至包含 fork 或者 clone 最新代码。也可以看到最近的 issue。
5. TensorFlow 在 Dockerhub 上维护的公开 Docker 镜像，网址为 <https://hub.docker.com/r/tensorflow/tensorflow/>。
6. TensorFlow 提供可下载的虚拟机，基于 Ubuntu 15.04 操作系统安装 TensorFlow。这样对于 Windows PC 用户更容易使用 TensorFlow 的 UNIX 版本。
7. Stack Overflow 上有 TensorFlow 标签的知识问答。随着 TensorFlow 日益流行，这个标签下的问答在不断增长，访问网址为：<http://stackoverflow.com/questions/tagged/TensorFlow>。
8. TensorFlow 非常灵活，应用场景广，最常用的是深度学习。为了理解深度学习的基础，数学知识和深度学习开发，Google 在在线课程 Udacity 上开课，网址为：<https://www.udacity.com/course/deep-learning--ud730>。
9. TensorFlow 也提供一个网站，让你可以可视化查看随着参数和样本数据集的变化对训练神经网络的影响，网址为：<http://playground.tensorflow.org>。
10. 深度学习开山祖师爷 Geoffrey Hinton 在 Coursera 上开课教授“机器学习中的神经网络”，网址为：<https://www.coursera.org/learn/neural-networks>。
11. 斯坦福大学提供在线课程“图像识别中卷积神经网络”及其详细的课件，网址为：<http://cs231n.stanford.edu/>。

参考

- ▶ Winters, D. https://docs.google.com/forms/d/1mUztU1K6_z31BbMW5ihXaYHlhBcbDd94mERe-8XHyoI/viewform

本PDF仅是样章,书籍版权归著者和出版社所有，未经允许不能在网上传播，如有需要，请尽量购买正版实体书，以表示对知识的尊重！实在有必要获取完整版本PDF，请联系QQ:2415996420.,谢谢！

2415996420